

DVS Technologies — AI Engineer with Cloud Foundations

Working Region for all labs: `ap-south-1` (Asia Pacific — Mumbai)

Level: Beginner to Intermediate

Module 1: AWS Networking — VPC, Subnets & CIDR

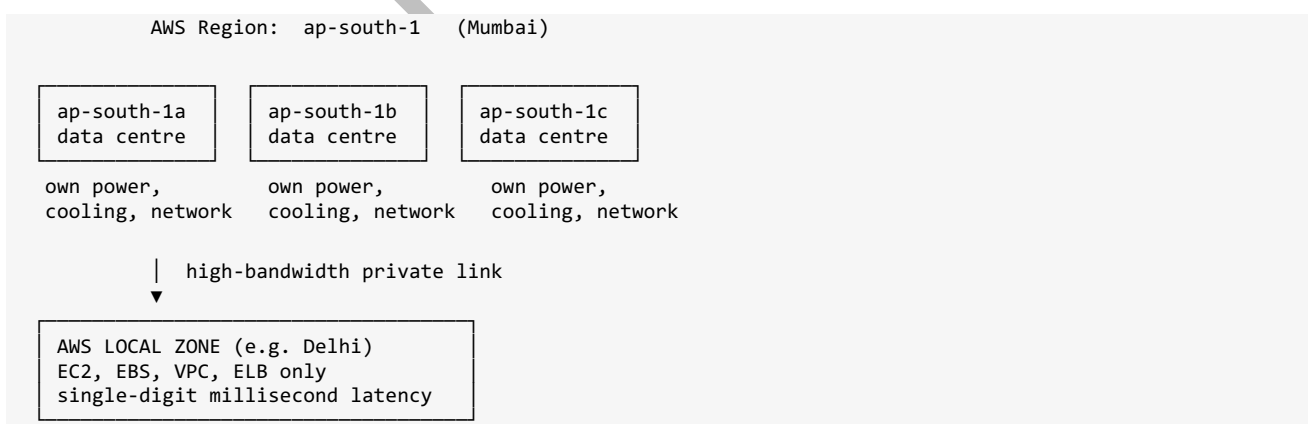
Networking is the foundation everything else sits on. If you understand the network, EC2 and RDS become simple. If you skip it, you will spend hours wondering why your web server "doesn't work."

1.1 The Foundation: Regions, Availability Zones & Local Zones

- An **AWS Region** is a separate geographic area — for example `ap-south-1` (Mumbai). Regions are fully independent of one another.
- An **Availability Zone (AZ)** is an isolated location *inside* a Region, backed by one or more discrete data centres with independent power, cooling, and networking. `ap-south-1` offers `ap-south-1a`, `ap-south-1b`, and `ap-south-1c`.
- Deploying across **two or more AZs** is the single most important high-availability decision you will make. An AZ can fail; a well-designed application survives it.
- **AWS Local Zones** extend core services (EC2, EBS, VPC, ELB) closer to large population centres to deliver **single-digit millisecond latency**. They are useful for latency-sensitive workloads (gaming, AR/VR, live media) and for **data residency** requirements, and they connect back to a parent Region over a high-bandwidth private link.

The AWS global footprint currently spans roughly **123 Availability Zones across 39 Regions**, with further expansion announced (including Saudi Arabia and Chile). Treat any specific count as a snapshot — always check the official Regions and AZs page for current numbers.

Diagram 1.1 — Region, Availability Zones and Local Zones



Read the diagram this way: each AZ has its own power, cooling, and networking, so a failure in `ap-south-1a` does not take down `ap-south-1b`. That is the whole reason every lab in this guide spreads subnets across 1a and 1b. A **Local Zone** is an extension of its *parent* Region and inherits the same APIs and IAM; a *different* Region shares nothing with `ap-south-1` unless you deliberately replicate to it.

1.2 IPv4 and CIDR Fundamentals

An **IPv4** address is a **32-bit** number used to identify a device on a network. **CIDR (Classless Inter-Domain Routing)**, introduced in 1993, is the flexible notation used to describe *ranges* of those addresses without the rigid old Class A/B/C boundaries.

Notation: an IP address followed by a slash and a prefix length — `192.168.1.0/24`.

Prefix meaning: the number after the slash is how many leading bits belong to the **network** portion. A `/24` fixes the first 24 bits as network, leaving $32 - 24 = 8$ bits for **hosts**.

The formula you must memorise:

Total IP addresses = $2^{(32 - \text{prefix length})}$

CIDR Block	Host Bits	Total IPs	Address Range
<code>192.168.1.0/32</code>	0	1	<code>192.168.1.0</code> only (a single host)
<code>192.168.1.0/24</code>	8	256	<code>192.168.1.0</code> – <code>192.168.1.255</code>
<code>192.168.1.0/23</code>	9	512	<code>192.168.1.0</code> – <code>192.168.2.255</code>
<code>10.0.0.0/16</code>	16	65,536	<code>10.0.0.0</code> – <code>10.0.255.255</code>
<code>0.0.0.0/0</code>	32	4,294,967,296	the entire IPv4 internet ("anywhere")

Why CIDR matters: it slowed IPv4 address exhaustion, shrank router tables, and lets network administrators create custom-sized subnets instead of wasting whole classes.

The 5 reserved addresses — a classic exam trap

AWS reserves **five IP addresses in every subnet**, so usable hosts are always `total - 5`. In a `10.0.0.0/24` subnet:

Address	Reserved for
<code>10.0.0.0</code>	Network address
<code>10.0.0.1</code>	VPC router
<code>10.0.0.2</code>	Amazon-provided DNS resolver
<code>10.0.0.3</code>	Reserved for future AWS use
<code>10.0.0.255</code>	Network broadcast (unused, but still reserved)

So a `/24` gives you **251 usable** addresses, and a `/28` — the smallest subnet AWS allows — gives you only **11**. The largest allowed VPC CIDR is `/16`.

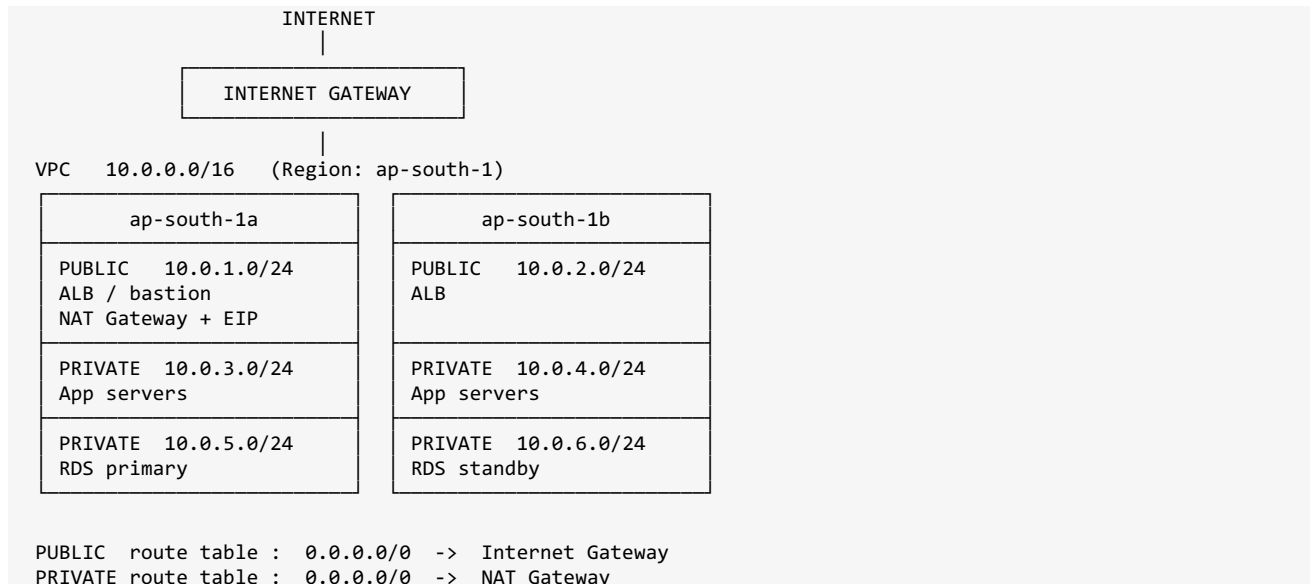
1.3 Amazon VPC and Its Core Components

Amazon VPC (Virtual Private Cloud) lets you provision a logically isolated section of the AWS Cloud in which you launch resources on a virtual network you define. You control your own IP address range, subnets, route tables, and gateways. It is the foundation of your AWS network infrastructure — everything from a single web server to a complex multi-tier architecture is built on it.

Component	Scope	Purpose
VPC	Spans all AZs in one Region	The isolated virtual network itself; defined by a CIDR block
Subnet	Lives entirely inside one AZ (cannot span AZs)	A slice of the VPC CIDR where resources are launched
Route Table	Associated with subnets	Rules that decide where outbound traffic goes
Internet Gateway	One per VPC	Two-way door to the public internet

Component	Scope	Purpose
NAT Gateway	Placed in a public subnet	Outbound-only internet access for private subnets
Security Group	Instance / ENI level	Stateful virtual firewall
Network ACL	Subnet level	Stateless subnet firewall

Diagram 1.3 — Anatomy of a production VPC



The single most important line in this diagram is **PUBLIC route table : 0.0.0.0/0 -> Internet Gateway**. Delete that one route and the "public" subnets instantly become private. Notice also that the private route table has **no** path to the IGW — outbound traffic detours through the NAT Gateway sitting in a *public* subnet, and S3 traffic bypasses it entirely through the free Gateway Endpoint.

With a /16 VPC this consumes a tiny fraction of the space, leaving plenty of room for future expansion. **Always leave room to grow** — a VPC CIDR cannot be shrunk, and resizing later means adding secondary CIDR blocks.

1.4 Public vs. Private Subnets

A subnet is not "public" because of a checkbox. **It is public only because its route table has a route to an Internet Gateway**. That single fact explains most beginner networking confusion.

Aspect	Public Subnet	Private Subnet
Route to IGW	Yes — 0.0.0.0/0 → igw-...	No
Inbound from internet	Possible (if SG/NACL allow)	Not possible directly
Outbound to internet	Direct, via IGW	Only via NAT Gateway
Public IP needed	Yes, for internet reachability	No
Typical residents	Load balancers, bastion hosts, public web servers, NAT Gateways	Application servers, databases , caches, internal services
Security posture	Exposed — harden carefully	Isolated by default

Best practice: put as little as possible in public subnets. A load balancer in public subnets fronting application servers in private subnets is the standard, and databases should *never* be publicly reachable.

1.5 Route Tables

A **route table** is a set of rules (**routes**) that determine where network traffic from a subnet or gateway is directed. Every subnet must be associated with exactly one route table.

- **Main route table:** created automatically with the VPC. It governs any subnet you don't explicitly associate elsewhere. **Leave it private** (no IGW route) so that a forgotten subnet fails closed rather than open.
- **Custom route tables:** create these to give different subnets different behaviour — for example one public table pointing at the IGW and one private table pointing at the NAT Gateway.
- Every route table implicitly contains a **local** route for the VPC CIDR, which is why all subnets in a VPC can talk to each other by default.
- Routing uses **most-specific-prefix-wins**: a route for `10.0.5.0/24` beats `0.0.0.0/0`.

1.6 Internet Gateway (IGW)

An **Internet Gateway** is a horizontally scaled, redundant, and highly available VPC component that enables communication between your VPC and the internet. It serves two purposes:

1. Providing a **target in your route tables** for internet-routable traffic (`0.0.0.0/0`).
2. Performing **network address translation (NAT)** for instances that have been assigned public IPv4 addresses.

Key facts:

- A VPC can have **only one IGW attached at a time**, and an IGW attaches to exactly one VPC.
- Without an IGW, your VPC is completely isolated from the internet.
- The IGW itself is **free**; you pay only for data transfer out.

Three things must all be true for an instance to be internet-reachable: an IGW attached to the VPC, a `0.0.0.0/0` route to that IGW in the subnet's route table, and a public IP (or Elastic IP) on the instance — plus permissive Security Group and NACL rules.

1.7 NAT Gateway

Resources in a private subnet often need **outbound** access — to download OS patches, call a third-party API, or reach an AWS service — without being exposed to **inbound** internet traffic. That is the job of **NAT (Network Address Translation)**.

Feature	NAT Gateway (recommended)
Management	Fully managed by AWS
Availability	Highly available within one AZ
Bandwidth	Scales automatically up to 100 Gbps
Patching / OS upkeep	None — AWS handles it
Security Groups	Not supported (use NACLs/route design)
Cost	~\$0.045–0.05 per hour + ~\$0.045 per GB processed
Free Tier eligible	No

Placement rules that trip students up:

- The NAT Gateway itself lives in a **public** subnet and requires an **Elastic IP**.
- The **private** subnet's route table gets `0.0.0.0/0 → nat-....`
- For real high availability, deploy **one NAT Gateway per AZ** — a single NAT Gateway becomes a cross-AZ single point of failure.

⚠ **Free Tier warning:** a NAT Gateway costs roughly ₹2,700–3,000 (~\$32–33) per month even when idle. Do not create one for practice unless you delete it the same day. Our labs deliberately avoid it.

1.8 VPC Endpoints — The Cheaper Path to S3

An **S3 Gateway Endpoint** is a route-table target inside your VPC that lets private subnets reach **Amazon S3** directly over the AWS private network.

	S3 Gateway Endpoint	NAT Gateway
Monthly cost	\$0	~\$32–33 per gateway
Data processing	\$0 per GB	~\$0.045 per GB
Network path	Private instance → local route table → S3. Never touches the public internet	Subnet → NAT GW → IGW → S3 public endpoint
Exposure	Stays entirely inside AWS	Extra hops and external exposure
Supports	S3 and DynamoDB (Gateway type)	Any internet destination

Heavy S3 traffic routed through a NAT Gateway drives cloud bills up fast. **If a private workload talks to S3 or DynamoDB, add a Gateway Endpoint — it is free and strictly more secure.** For other AWS services, the equivalent is an **Interface Endpoint** (AWS PrivateLink), which does carry an hourly charge.

Diagram 1.8 — Two paths to the same bucket

```
PATH A    through a NAT Gateway    (costs money, leaves AWS)

EC2 in    -> NAT Gateway -> Internet -> public -> S3
my-infra-private-1a    $0.045/GB    Gateway    internet
                        + $32/month    (in my-infra-public-1a)

PATH B    through an S3 Gateway Endpoint    (free, stays inside AWS)

EC2 in    -> S3 Gateway Endpoint -> S3
my-infra-private-1a    $0.00/GB, $0/month
                        never touches the internet
```

Path A costs money at every hop and briefly exposes traffic to the public internet. **Path B** is free, shorter, and never leaves AWS. Same bucket, same API calls, very different bill and threat surface — this is one of the highest-value 30-second changes you can make to a real workload.

1.9 Security Groups vs. Network ACLs

Security inside a VPC is enforced at two layers. Use both; they are complementary, not alternatives.

Security Groups (instance level)

A **Security Group** acts as a virtual firewall for your EC2 instances, controlling inbound and outbound traffic. It attaches to an **ENI (Elastic Network Interface)** — up to **5 Security Groups per ENI**.

- **Stateful:** if a request is allowed out, the response is automatically allowed back in (and vice versa). You never write "return traffic" rules.
- **Allow rules only:** you cannot create a Deny rule. Everything not explicitly allowed is denied.
- **Default behaviour:** a new Security Group **blocks all inbound** and **allows all outbound** traffic.
- Rules from multiple attached groups are **aggregated** into one effective policy.
- A Security Group can reference **another Security Group** as its source — the cleanest way to say "only my web tier may reach my database."

Practical example — web server Security Group:

Direction	Protocol	Port	Source / Destination	Description
Inbound	TCP	443	0.0.0.0/0	Allow HTTPS from anywhere
Inbound	TCP	80	0.0.0.0/0	Allow HTTP from anywhere
Inbound	TCP	22	<your-IP>/32	SSH from your IP only — never 0.0.0.0/0
Outbound	All	All	0.0.0.0/0	Allow all outbound traffic

Guidance: do 95% of your work in Security Groups.

Diagram 1.9 — The journey of one packet through both layers

```

REQUEST IN:
  Client -> IGW -> SECURITY GROUP -> EC2 port 80

RESPONSE OUT:
  EC2 -> SECURITY GROUP : allowed automatically

```

Hands-On Lab 1: Free Tier Guide

Goal: Build the class assignment network in Mumbai — VPC 10.0.0.0/24 split into four subnets of 64 IPs each across two AZs.

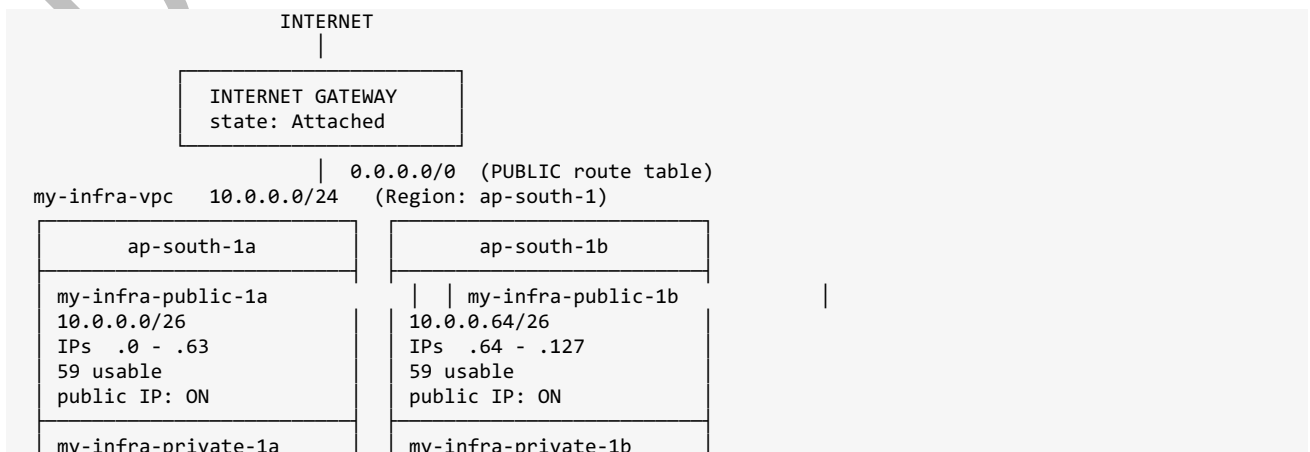
Cost: ₹0 / \$0. VPCs, subnets, route tables, Internet Gateways, Security Group. We will not create a NAT Gateway or Elastic IP.

Your subnet plan

10.0.0.0/24 = 256 addresses. Dividing into four equal blocks means four /26 subnets ($2^{(32-26)} = 64$ IPs each, **59 usable** after AWS's 5 reserved).

Subnet Name	Type	AZ	CIDR	Range	Usable IPs
my-infra-public-1a	Public	ap-south-1a	10.0.0.0/26	.0 – .63	59
my-infra-public-1b	Public	ap-south-1b	10.0.0.64/26	.64 – .127	59
my-infra-private-1a	Private	ap-south-1a	10.0.0.128/26	.128 – .191	59
my-infra-private-1b	Private	ap-south-1b	10.0.0.192/26	.192 – .255	59

Diagram Lab 1 — Your lab network in ap-south-1



```
10.0.0.128/26
IPs .128 - .191
59 usable
no 0.0.0.0/0 route
```

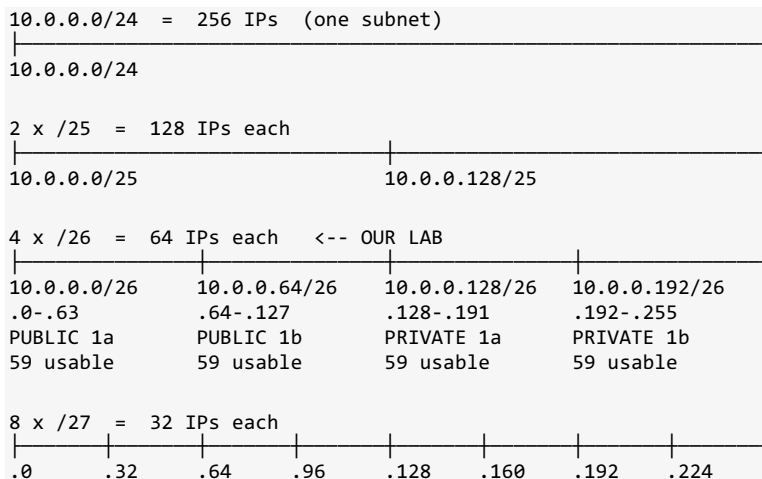
```
10.0.0.192/26
IPs .192 - .255
59 usable
no 0.0.0.0/0 route
```

Lab 3 -> EC2 web server goes in my-infra-public-1a

Lab 4 -> RDS DB subnet group = my-infra-private-1a + my-infra-private-1b

Diagram Lab 1b — Splitting a /24 into four /26 subnets

Each step down the ladder **halves** the block. Adding 1 to the prefix doubles the number of subnets and halves their size:



Notice the boundaries are never arbitrary: a /26 can only start at .0, .64, .128, or .192. If you ever try to create 10.0.0.50/26, the console will reject it — the network address must fall on a multiple of the block size.

Step-by-step (AWS Management Console)

- Set your Region.** Top-right Region selector → **Asia Pacific (Mumbai) ap-south-1**. Confirm this before every lab; resources created in the wrong Region are invisible in the right one.
- Search for and open the **VPC** console. Click **Create VPC**.
- Choose **VPC and more** (the wizard that builds subnets and route tables for you).
 - Name tag auto-generation:** my-infra
 - IPv4 CIDR block:** 10.0.0.0/24
 - IPv6 CIDR block:** No IPv6 CIDR block
 - Tenancy:** Default
 - Number of Availability Zones (AZs):** 2
 - Expand **Customize AZs** and select ap-south-1a and ap-south-1b.
 - Number of public subnets:** 2
 - Number of private subnets:** 2
 - Expand **Customize subnets CIDR blocks** and enter the four /26 values from the table above.
 - NAT gateways:** None — *this is the one setting that can cost you real money. Leave it at None.*
 - VPC endpoints:** S3 Gateway — this is free
 - DNS options:** leave both **Enable DNS hostnames** and **Enable DNS resolution** checked.
- Click **Create VPC** and watch the resource map render.
- Verify your work** — this is the real learning:
 - Subnets** → confirm four subnets, correct CIDRs, correct AZs.
 - Route tables** → open the **public** route table, check the **Routes** tab for 0.0.0.0/0 → igw-..., and check the **Subnet associations** tab lists your two public subnets.

- **Route tables** → open a **private** route table and confirm there is **no** `0.0.0.0/0` route. This is what makes it private.
 - **Internet gateways** → confirm state is **Attached**.
6. **Enable auto-assign public IP** on the public subnets (used in Lab 3): select `my-infra-public-1a` → **Actions** → **Edit subnet settings** → check **Enable auto-assign public IPv4 address** → **Save**. Repeat for `my-infra-public-1b`.
 7. Create a Security Group for later use: **VPC** → **Security groups** → **Create security group**.
 - **Name:** `lab-web-sg`, **VPC:** your `my-infra-vpc`
 - **Inbound rules:** HTTP from `Anywhere-IPv4`; SSH from `0.0.0.0/0`
 - Leave the default outbound rule intact → **Create security group**.

Clean-up (optional)

Everything in this lab is free and is used by Labs 3 and 4, so **keep it**. If you do want to remove it, delete resources in this order: EC2 instances → RDS databases → NAT Gateways (none here) → Subnets → Route tables → Detach and delete the IGW → Delete the VPC. The console's **Delete VPC** action will offer to remove most dependencies for you.

Module 1 Key Takeaways

- **IPv4 is 32 bits; CIDR describes ranges.** $\text{Total IPs} = 2^{(32 - \text{prefix})}$, and AWS reserves **5** addresses per subnet.
- A **VPC** spans a Region; a **subnet** lives in exactly **one AZ**. Spread across AZs for availability.
- A **subnet is public only because its route table points to an Internet Gateway**. Nothing else makes it public.
- **NAT Gateway** = managed, highly available, and **not free** (~\$32/month plus per-GB). Place it in a public subnet with an Elastic IP; the private route table points to it.
- Use a **free S3 Gateway Endpoint** instead of routing S3 traffic through NAT — cheaper *and* more secure.
- **Security Groups** are stateful, allow-only, instance-level; **NACLs** are stateless, allow-and-deny, subnet-level.

Module 2: Identity and Access Management (IAM)

Networking decides *where* traffic can go. **IAM decides who is allowed to do what**. It is the most security-critical service in AWS, and it is completely free.

2.1 What IAM Does

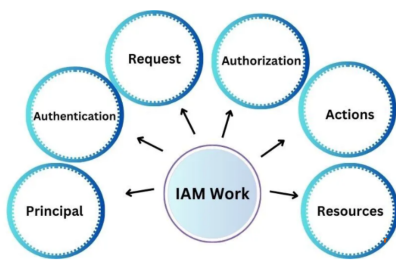
AWS Identity and Access Management (IAM) is a web service that helps you securely control access to AWS resources. With IAM you manage permissions that determine which AWS resources users can access. IAM provides the infrastructure necessary to control **authentication** and **authorization** for your AWS accounts.

You use IAM to control two primary functions:

- **Authentication** — controlling **who can sign in**.
- **Authorization** — controlling **what those signed-in users have permission to do**.

IAM is a **global** service: identities are not tied to a Region, even though the resources they act on usually are.

2.2 The IAM Request Workflow



When an entity interacts with AWS, the process encompasses six key elements:

1. **Principal** — an entity capable of executing actions on an AWS resource. This could be a **user**, a **role**, or an **application**.
2. **Authentication** — validating the principal's identity. The principal must furnish credentials or keys.
3. **Request** — the principal sends a request to AWS specifying the desired **action** and the **target resource**, along with context such as source IP, time, and whether MFA was used.
4. **Authorization** — **by default, all resource access is denied**. IAM authorizes the request only if every aspect aligns with a matching policy.
5. **Actions** — the specific operations performed to **view, create, modify, or delete** a resource (for example `s3:GetObject`, `ec2:RunInstances`).
6. **Resources** — the specific AWS objects the actions are performed on, identified by **ARN (Amazon Resource Name)**.

Diagram 2.2 — The IAM request flow

1	PRINCIPAL	user, role or application
2	AUTHENTICATION	password + MFA, access keys, or temporary role credentials
3	REQUEST	action + resource + context
4	AUTHORIZATION	evaluate every applicable policy - explicit Deny found -> DENIED (Deny always wins) - no matching Allow -> DENIED (implicit deny) - explicit Allow, no Deny -> continue
5	ACTION	view / create / modify / delete
6	RESOURCE	identified by its ARN

The habit to build from this diagram: when a permission problem appears, walk the six steps in order. Most "IAM is broken" tickets are really step 3 (the ARN in the policy doesn't match the resource being touched) or step 4 (an explicit Deny in a broader policy is silently overriding your Allow).

2.3 Users, Groups & Roles

IAM Users

IAM users function as distinct identities with associated credentials inside the AWS ecosystem. They can represent actual individuals or applications, and **each IAM user is exclusively linked to one AWS account**.

On creation, a new IAM user has **no permissions at all** — an intentional security measure. This one-to-one model lets administrators tailor permissions individually, aligning perfectly with the **principle of least privilege**.

Two credential types, for two very different purposes:

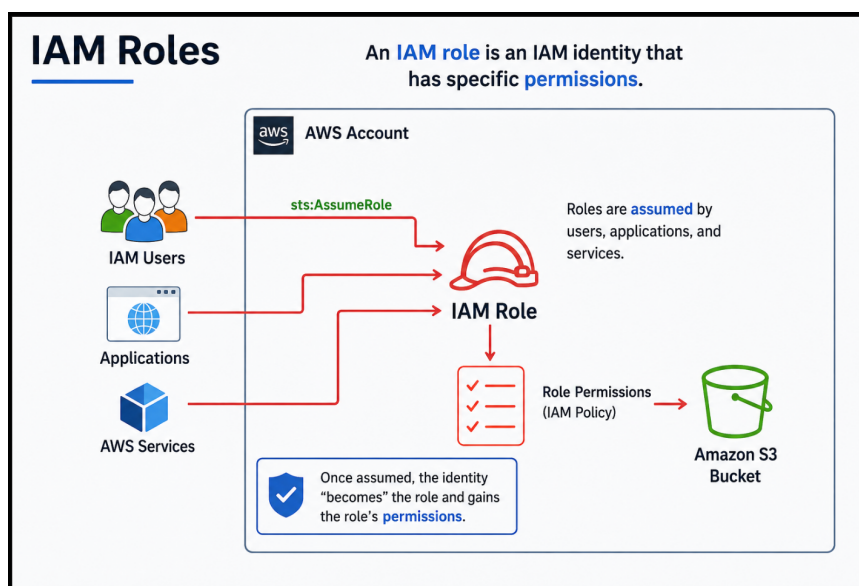
Credential	Used for
Console password	Human sign-in to the AWS Management Console
Access key ID + secret access key	Programmatic access (CLI, SDK, API)

IAM Groups

An assemblage of IAM users forms an **IAM group**. Groups offer a streamlined way to manage permissions for multiple users concurrently: any permission allocated to the group extends automatically to every user inside it. When a new user is added to a group they effortlessly inherit all its policies, reducing administrative overhead and speeding up onboarding.

- Attach policies to **groups**, not to individual users — this is the habit that keeps large accounts manageable.
- Groups **cannot** be nested, and a group is **not** an identity (it cannot be a policy principal).

IAM Roles



An **IAM role** is a set of permissions defining permissible actions for an entity. Unlike standard users, roles provide **temporary credentials** and can be **assumed** by various entities as needed. Roles have **no long-term password or access key** — which is precisely what makes them the secure choice.

Roles are invaluable for:

- Granting a **mobile app** or an **EC2 instance** access **without storing sensitive keys**.
- Allowing users authenticated **outside of AWS** (e.g., via Google or Facebook, or corporate SAML/SSO) to access resources — **federation**.
- Providing **temporary access to third parties** such as consultants or auditors.
- **Cross-account access** between AWS accounts.

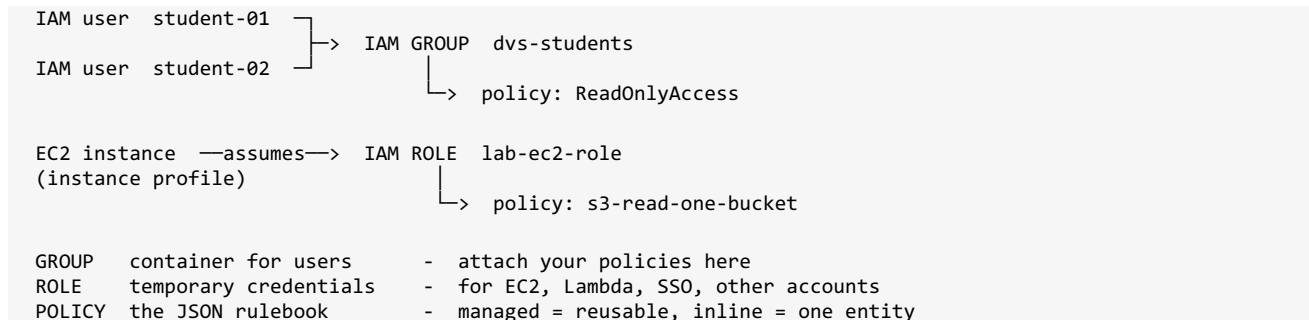
Every role has two policy halves:

Policy	Question it answers
Trust policy	<i>Who is allowed to assume this role? (e.g., the EC2 service)</i>
Permissions policy	<i>What can the role do once assumed?</i>

	IAM User	IAM Role
Credentials	Long-term (password / access keys)	Temporary , auto-rotated

	IAM User	IAM Role
Associated with	One specific person or app	Assumed by any trusted entity
Best for	Human console access	Applications, EC2, Lambda, federation, cross-account
Security posture	Keys can leak	Nothing to leak — strongly preferred

Diagram 2.3 — How identities, groups, roles and policies fit together



Two rules of thumb are visible in this picture. Permissions should flow **user → group → policy** (attach to the group, never to the person), and anything that is not a human should reach AWS by **assuming a role**, never by holding a key. What a role is allowed to *do* comes from its **permissions policy**; who is allowed to *assume* it comes from its **trust policy**.

2.4 Understanding IAM Policies

Policies are the mechanisms that permit or deny a connection based on specified rules. Stored as **JSON documents**, policies articulate **who** has access, **what actions** they can take, and **which resources** they can touch.

Anatomy of a JSON policy document

- **Version** — the policy language version (always use "2012-10-17").
- **Statement** — an array containing one or more statements defining the permissions.
- **Sid (optional)** — a statement identifier for readability.
- **Effect** — whether the statement **Allows** or **Denys** access.
- **Action** — the exact operation(s) permitted, e.g. "s3:*" for all S3 actions.
- **Resource** — the AWS resources, via **ARNs**, to which the actions apply.
- **Condition (optional)** — extra constraints such as requiring MFA or a source IP range.

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ReadOneBucket",
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:ListBucket"
      ],
      "Resource": [
        "arn:aws:s3:::dvs-student-bucket",
        "arn:aws:s3:::dvs-student-bucket/*"
      ]
    },
    {
      "Sid": "RequireMfaForDeletes",
      "Effect": "Deny",
      "Action": "s3:DeleteObject",
      "Resource": "arn:aws:s3:::dvs-student-bucket/*",
      "Condition": {

```

```

    "BoolIfExists": { "aws:MultiFactorAuthPresent": "false" }
  }
}
]
}

```

Read that closely: listing a bucket needs the **bucket ARN**, while reading objects needs the **/* object ARN**. Forgetting the **/*** is the most common IAM policy bug beginners hit.

Roles vs. Policies — a distinction students often blur

	IAM Role	IAM Policy
What it is	An identity that can be assumed	A JSON document of permissions
Grants access by itself?	Only once a policy is attached	Only once attached to a user, group, or role
Analogy	The job badge you temporarily wear	The rulebook printed on the badge
Has credentials?	Yes — temporary ones	No

2.5 Managed vs. Inline Policies

Organizations utilize two distinct types of policies to govern access control:

Feature	Managed Policies	Inline Policies
Scope	Autonomous, stand-alone policies	Embedded directly into a singular entity
Attachment	Can be attached to multiple users, groups, or roles	Intricately tied to the single entity it was created for
Origin	AWS-managed (default) or Customer-managed	Always custom-created for that specific entity
Advantage	Centrally administered; ensures consistency across entities	Highly granular control without affecting other entities
Disadvantage	Broader scope requires careful auditing	Increased administrative complexity at scale
Lifecycle	Independent; supports versioning and rollback	Deleted when the entity is deleted

Recommendation: default to **customer-managed policies**. Use **AWS-managed** policies to get started quickly (they are convenient but often broader than you need), and reserve **inline** policies for genuine one-off exceptions where the permission must never be reused.

2.6 Policy Evaluation and Least Privilege

Internalise these rules and IAM stops being mysterious:

1. **Implicit deny** — everything is denied by default.
2. An **explicit Allow** in any applicable policy grants access.
3. An **explicit Deny always wins**, no matter what else allows it.
4. **Least privilege** — grant only the permissions required, then widen if genuinely needed. Never start with `"Action": "*" and plan to tighten later; that plan never survives contact with a deadline.`

Tools that help: use the **IAM Policy Simulator** to test before deploying, **Access Analyzer** to find over-permissive access, and **Last accessed** data to trim unused permissions.

Hands-On Lab 2: Free Tier Guide

Goal: Create a group, a user, and an EC2 service role you will attach to your instance in Lab 3.

Cost: ₹0 / \$0 — IAM is always free. Users, groups, roles, and policies cost nothing, in any quantity. There is nothing to clean up for billing reasons (though tidy accounts are good practice).

Part A — Create a group and a user

1. Open the **IAM** console. (No Region selector — IAM is global.)
2. **Access management** → **User groups** → **Create group**.
 - **Name:** `dvs-students`
 - **Attach permissions policies:** search and select **ReadOnlyAccess** (AWS-managed) → **Create user group**.
3. **Access management** → **Users** → **Create user**.
 - **User name:** `student-01`
 - Check **Provide user access to the AWS Management Console**
 - Select **I want to create an IAM user**
 - **Console password:** do not keep Autogenerated, and uncheck **Users must create a new password at next sign-in** checked → **Next**
4. **Permissions options:** choose **Add user to group** → tick `dvs-students` → **Next** → **Create user**.
6. **Test in a private browser window:** sign in as `student-01` using the account sign-in URL. Confirm you can *view* EC2 and VPC resources but get an **access denied** error if you try to launch an instance. **That error is a success** — it proves least privilege is working.
7. **Recommended in live env:** back in your admin session, open `student-01` → **Security credentials** → **Assign MFA device** and register an authenticator app.

Part B — Create a customer-managed policy

1. **IAM** → **Policies** → **Create policy** → **JSON** tab.
2. Paste the S3 read-only policy from §2.4 (change the bucket name to something unique, e.g. `dvs-<yourname>-bucket`).
3. **Next** → **Name:** `dvs-s3-read-one-bucket` → **Create policy**.

Part C — Create an EC2 service role (used in Lab 3)

1. **IAM** → **Roles** → **Create role**.
2. **Trusted entity type:** **AWS service** → **Use case:** **EC2** → **Next**.
3. **Add permissions:** select **AmazonSSMManagedInstanceCore** (this later lets you connect to your instance through **Session Manager** with **no SSH key and no open port 22** — the modern, safer pattern). Optionally also attach your `dvs-s3-read-one-bucket` policy.
4. **Role name:** `lab-ec2-role` → **Create role**.
5. Open the new role and read the **Trust relationships** tab. Notice `"Service": "ec2.amazonaws.com"`

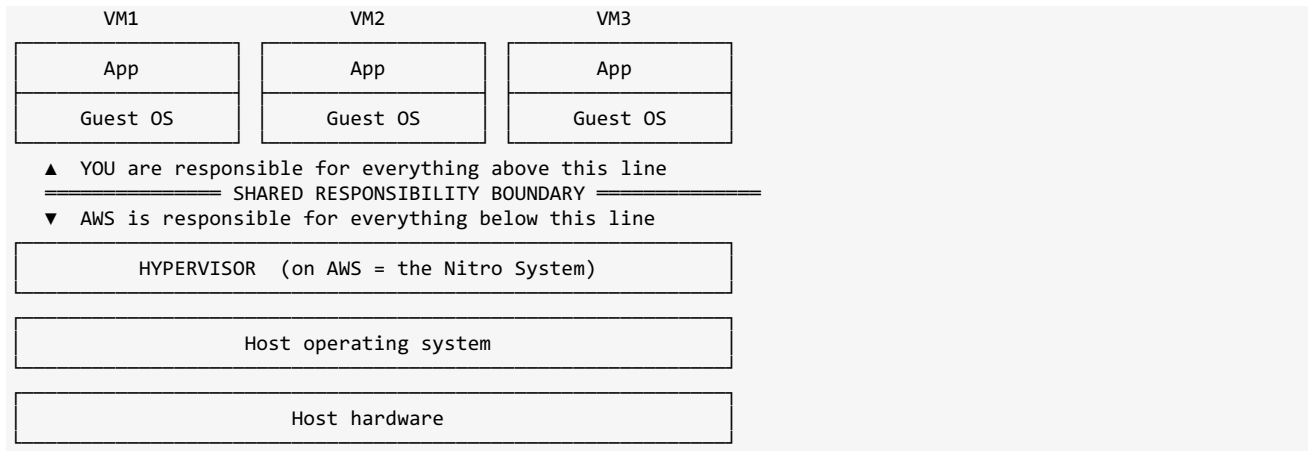
Attaching a role to a running instance (as shown in class): **EC2** → **Instances** → select the instance → **Actions** → **Security** → **Modify IAM role**. Roles can be attached or swapped without stopping the instance.

Module 3: Elastic Compute Cloud (EC2)

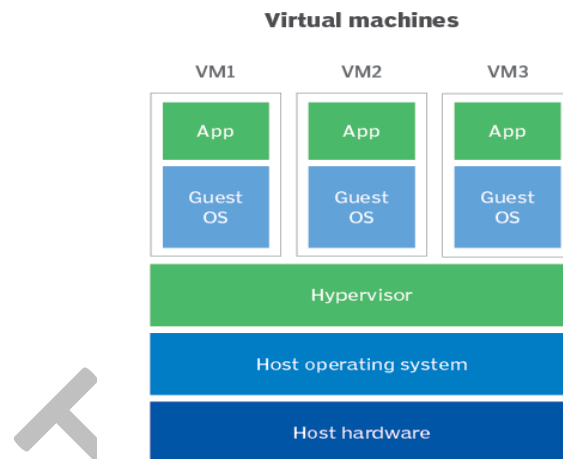
Amazon EC2 provides resizable compute capacity — virtual servers you launch in minutes and pay for by the second. It is where your network and IAM knowledge finally becomes something you can browse to.

3.1 From Virtualization to EC2

Diagram 3.1 — The virtualization stack



The same stack as drawn in class:



Virtual machines running on a hypervisor above the host OS and host hardware

Each EC2 instance is one of those VMs. You patch the guest OS, configure the firewall, and secure the application; AWS owns the hypervisor downward. Everything you will be asked about the **Shared Responsibility Model** can be answered by locating the item above or below that boundary line.

3.2 Anatomy of an EC2 Instance

Every launch is a set of decisions:

Choice	What it means	Free Tier guidance
AMI (Amazon Machine Image)	The OS template — Amazon Linux 2023, Ubuntu, Windows...	Pick one labelled Free tier eligible
Instance type	The CPU/RAM/network shape, e.g. t3.micro	t2.micro or t3.micro only
Key pair	The SSH credential	Create one, or use Session Manager instead
Network	VPC + subnet + public IP	Your my-infra-vpc, a public subnet
Security Group	The instance firewall	lab-web-sg from Lab 1
Storage	EBS root volume	≤ 30 GB of gp3 total

Choice	What it means	Free Tier guidance
IAM instance profile	The role the instance assumes	lab-ec2-role from Lab 2
User data	A script that runs at first boot	Our Apache bootstrap

Instance families at a glance:

<https://aws.amazon.com/ec2/instance-types/>

<https://docs.aws.amazon.com/ec2/latest/instancetypes/gp.html>

Instance Type (Prefix)	Category	Usage / Target Workloads	Examples
T Type	General Purpose (Burstable)	Websites, dev/test environments, and microservices. These provide a baseline level of CPU performance with the ability to "burst" above the baseline when needed.	t2.micro, t3.large, t4g.small
M Type	General Purpose (Fixed)	Backend servers, small/mid-size databases, and enterprise applications. They provide a consistent balance of compute, memory, and network resources.	m5.large, m7i.xlarge, m8g.large
Mac Type	General Purpose (macOS)	Developing, building, testing, and signing applications for Apple platforms (macOS, iOS, iPadOS, tvOS, watchOS).	mac1.metal, mac2.metal
C Type	Compute Optimized	Batch processing, media transcoding, high-performance web servers, and scientific modeling. Ideal for compute-bound applications that benefit from high-performance processors.	c5.xlarge, c6g.2xlarge, c7i.large
R Type	Memory Optimized	Relational databases, NoSQL databases, in-memory caches (like Redis), and real-time big data analytics.	r5.large, r6i.4xlarge, r7g.xlarge
X / z Type	High Memory Optimized	Enterprise-scale workloads, SAP HANA, massive in-memory databases, and electronic design automation (EDA).	x2iezn.2xlarge, z1d.large
I Type	Storage Optimized (High I/O)	Transactional NoSQL databases (Cassandra, MongoDB), Elasticsearch, and data warehousing. They offer extremely high sequential read/write access to local storage.	i3.large, i4i.16xlarge
D / H Type	Storage Optimized (Dense)	MapReduce/Hadoop environments, distributed file systems, and massive data lakes. Best for workloads requiring massive amounts of sequential disk I/O.	d3.8xlarge, h1.2xlarge

Instance Type (Prefix)	Category	Usage / Target Workloads	Examples
P Type	Accelerated (GPU - Compute)	Machine learning model training, deep learning, autonomous vehicle research, and high-performance computing (HPC).	p3.2xlarge, p4d.24xlarge
G Type	Accelerated (GPU - Graphics)	Graphics-intensive applications, 3D rendering, video encoding, and machine learning inference.	g4dn.xlarge, g5.2xlarge
Inf / Trn	Accelerated (AWS Silicon)	Highly cost-effective machine learning inference (AWS Inferentia) and machine learning model training (AWS Trainium).	inf2.xlarge, trn1.2xlarge

Purchasing options: **On-Demand** (pay per second, no commitment — what Free Tier uses),

Savings Plans / Reserved (up to ~72% off for a 1–3 year commitment),

Spot (up to ~90% off but interruptible), and

Dedicated Hosts/Instances (compliance isolation, expensive — For highly secure workloads).

3.3 Storage: EBS Volumes

Amazon EBS (Elastic Block Store) provides network-attached **block storage** — think of it as a virtual hard disk. This is the "block storage" column from your storage comparison:

	File storage	Block storage	Object storage
Basic idea	Files & folders	Raw disk blocks	Objects + metadata
Structure	Hierarchical folders	No filesystem by itself	Flat namespace / buckets
Access	SMB, NFS	Disk / SAN protocols	HTTP / API
Typical use	Shared files	Databases, VMs, OS disks	Backups, media, data lakes
Performance	Good	Very high / low latency	Usually higher latency
Scaling	Moderate	Moderate	Massive
AWS example	Amazon EFS / FSx	Amazon EBS	Amazon S3

Key EBS facts:

- A volume is tied to **one AZ**; **snapshots** (stored in S3) are how you move or back up data across AZs and Regions.
- **gp3** is the modern default general-purpose SSD — cheaper than gp2 with baseline 3,000 IOPS.
- **Delete on termination** is enabled by default for the root volume. Leaving it on prevents orphaned volumes quietly billing you; turn it off only when you deliberately want the data to survive.
-  **A detached EBS volume still costs money.** Deleting an instance without deleting stray volumes is the #1 source of surprise student bills.

3.4 Key Pairs and Secure Access

A **key pair** is public-key cryptography for instance login: AWS stores the **public** key, you download the **private** key (.pem) exactly once.

- Store the `.pem` safely and run `chmod 400 my-key.pem` — SSH refuses world-readable keys.
- **If you lose the private key, you cannot recover it.** Recovery means detaching the root volume or rebuilding.
- Restrict SSH (port 22) to **My IP**, never use `0.0.0.0/0` for stricter policy

3.5 User Data and Instance Metadata (IMDSv2)

User data is a script passed at launch that runs **once, as root, on first boot** — the simplest form of bootstrapping.

Instance metadata is information about the instance itself, retrieved from the link-local address `169.254.169.254`.

IMDSv2 is the secure, session-oriented version: you first **PUT** for a token, then send that token with every request. It defends against SSRF attacks that plagued IMDSv1, and **you should always use it**.

Here is the class bootstrap script, annotated:

```
#!/bin/bash
#!/bin/bash

# Update the system and install necessary packages
yum update -y
yum install -y httpd

# Start the Apache server
systemctl start httpd
systemctl enable httpd

# -----
# Get EC2 instance metadata using IMDSv2
# -----

TOKEN=$(curl -sX PUT \
  -H "X-aws-ec2-metadata-token-ttl-seconds: 21600" \
  http://169.254.169.254/latest/api/token)

INSTANCE_ID=$(curl -s \
  -H "X-aws-ec2-metadata-token: $TOKEN" \
  http://169.254.169.254/latest/meta-data/instance-id)

AVAILABILITY_ZONE=$(curl -s \
  -H "X-aws-ec2-metadata-token: $TOKEN" \
  http://169.254.169.254/latest/meta-data/placement/availability-zone)

# -----
# Create the index.html file
# -----

cat > /var/www/html/index.html <<EOF
<!DOCTYPE html>
<html>
<head>
  <title>EC2 Instance</title>
</head>

<body style="font-family:Arial; background:#f4f6f8;">

<div style="max-width:600px; margin:50px auto; background:#fff; padding:30px; border-radius:8px; text-align:center; box-shadow:0 2px 8px rgba(0,0,0,0.1);">

  <h2 style="color:#2c3e50; margin-bottom:25px;">
    🌟 Daily Motivation for my Agentic AI Students.. - Subrat
  </h2>

  <p style="font-size:18px; color:#555; font-style:italic;">
    "Success is the sum of small efforts, repeated day in and day out."
  </p>

  <p style="color:#777; margin-bottom:30px;">
    - Robert Collier
  </p>
  <hr style="border:none; border-top:1px solid #ddd; margin:25px 0;">
  <p style="font-size:18px; color:#555; font-style:italic;">
    "The future depends on what you do today."
```

```

</p>
<p style="color:#777;">
  - Mahatma Gandhi
</p>
<hr style="border:none; border-top:1px solid #ddd; margin:25px 0;">

<h3 style="color:#2c3e50;">EC2 Instance Details</h3>
<p>
  <strong>Instance ID:</strong><br>
  $INSTANCE_ID
</p>
<p>
  <strong>Availability Zone:</strong><br>
  $AVAILABILITY_ZONE
</p>
</div>

</body>
</html>
EOF

# Check the httpd service is correctly set up
systemctl enable httpd
systemctl restart httpd

```

Why print the instance ID and AZ? Because when you later put two instances behind a **load balancer**, refreshing the page and watching the ID change is the clearest possible proof that traffic is being distributed across AZs.

Also remember user data runs **only on the first boot** by default.

Note*:

IMDSv2 stands for **Instance Metadata Service Version 2**. It is an on-instance component in AWS that allows your EC2 instances to access data about themselves (like their IP address, VPC details, and IAM role temporary credentials) from a special, non-routable IP address (169.254.169.254).

3.6 Instance Profiles — Giving EC2 an IAM Role

When an application on EC2 needs to call AWS (read an S3 bucket, write to DynamoDB), **do not store access keys on the instance**. Attach an **IAM role** via an **instance profile**

Console path (as demonstrated in class): EC2 → Instances → select instance → Actions → Security → Modify IAM role → choose the role → Update IAM role. No restart required.

Diagram 3.6 — Attaching a role in the console

The exact console path, as demonstrated in class:

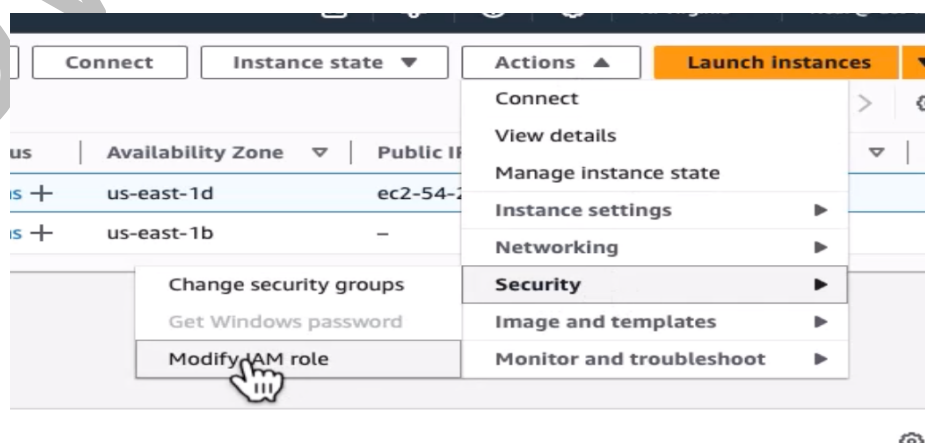
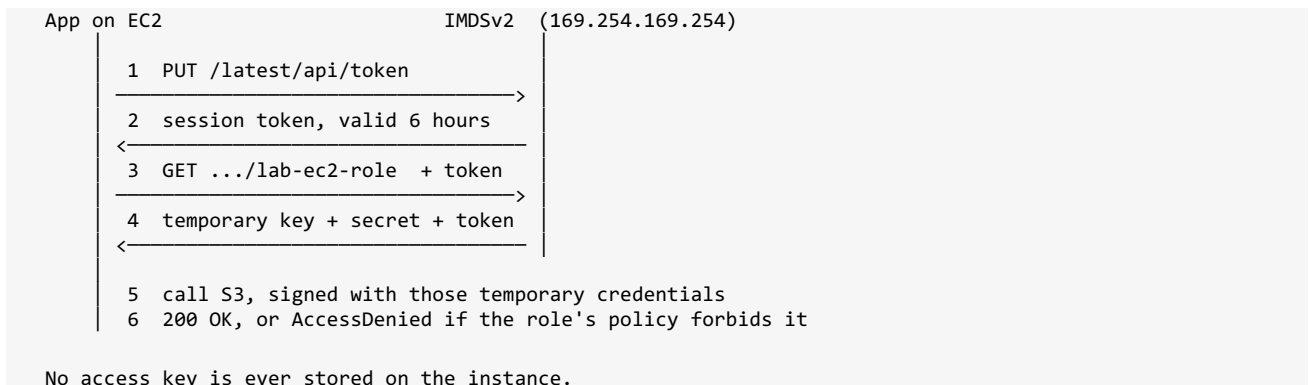


Diagram 3.6b — What actually happens when your code calls AWS

Why this matters: there is no access key on the instance, so there is no access key to leak, commit to Git, or rotate manually. This is the single biggest security win available to you on EC2, and it costs nothing.

Hands-On Lab 3: Free Tier Guide

Goal: Launch a Free Tier web server in your `my-infra-vpc` public subnet and reach it from a browser.

🎯 Free Tier settings — the ones that matter

Setting	Choose exactly this	Why
Region	ap-south-1 (Mumbai)	Consistency with Labs 1 & 4
AMI	Amazon Linux 2023, marked Free tier eligible	The <code>yum/systemctl</code> script above assumes Amazon Linux
Instance type	t2.micro or t3.micro — must show the Free tier eligible badge	750 instance-hours/month . Anything larger bills immediately
Storage	8 GB gp3, keep Delete on termination = Yes	Free Tier covers 30 GB of EBS total
Elastic IP	Do not allocate one	An unattached Elastic IP is billable
Detailed monitoring	Leave unchecked	1-minute CloudWatch metrics cost extra
Instances running	One	750 hours = one instance for a full month. Two instances = ~1,500 hours = charges

Step-by-step (AWS Management Console)

1. Confirm the Region is **Asia Pacific (Mumbai)** `ap-south-1`.
2. **EC2 → Instances → Launch instances**.
3. **Name and tags:** `lab-web-01`.
4. **Application and OS Images:** **Amazon Linux 2023 AMI** — verify the **Free tier eligible** label.
5. **Instance type:** `t3.micro` (or `t2.micro`) — verify the **Free tier eligible** label.
6. **Key pair:**
 - **Create new key pair** → name `lab-key`, type **RSA**, format **.pem** → download and `chmod 400` it.
7. **Network settings → Edit:**
 - **VPC:** `my-infra-vpc`
 - **Subnet:** `my-infra-public-1a`

- **Auto-assign public IP: Enable**
 - **Firewall (security groups): Select existing security group** → lab-web-sg
8. **Configure storage:** 8 GiB, gp3. Keep total EBS across your account under **30 GB**.
9. **Advanced details:**
- **IAM instance profile:** lab-ec2-role
 - **Metadata version: V2 only (token required)** — enforces IMDSv2
 - **User data:** paste the bootstrap script from §3.5
10. Review the **Summary** panel — **Number of instances must be 1** — then **Launch instance**.
11. Wait for **Instance state: Running** and **Status checks: 2/2 passed** (about 1–2 minutes).
12. **Test it:** copy the **Public IPv4 address** and open `http://<public-ip>` in a browser. You should see the motivation page with your instance's **Instance ID** and **Availability Zone**.
- Use `http://`, not `https://` — we never installed a TLS certificate.
 - If the browser hangs: check the SG allows **HTTP (80)** from `0.0.0.0/0`, the subnet's route table has `0.0.0.0/0` → `igw-...`, and the instance actually has a public IP.
13. **Optional exercise:** launch a second instance in `my-infra-public-1b` and compare the AZ shown on each page. **Terminate it immediately afterwards** — two instances burn Free Tier hours twice as fast.

Lab 3 Clean-Up Steps — DO NOT SKIP

Read this section before you close your laptop. A running instance bills every hour, whether or not anyone is looking at it. Free Tier hours are consumed silently and the invoice arrives a month later.

1. **Terminate the instance.** EC2 → **Instances** → select lab-web-01 → **Instance state** → **Terminate (delete) instance** → confirm. State becomes **Shutting-down**, then **Terminated**.
 - "Stopped" is not "terminated." A stopped instance stops CPU charges but **its EBS volume keeps billing**.
2. **Verify no EBS volumes survive.** EC2 → **Elastic Block Store** → **Volumes**. Any volume in state **Available** (i.e., unattached) → **Actions** → **Delete volume**. This is the most commonly missed step.
3. **Release any Elastic IPs.** EC2 → **Network & Security** → **Elastic IPs**. Any address listed → **Actions** → **Release Elastic IP addresses**. An EIP not attached to a running instance is billed hourly.
4. **Delete unused snapshots and AMIs.** EC2 → **Snapshots** and EC2 → **AMIs** — if you created any, deregister the AMI and delete its snapshots.
5. **Delete the key pair** if you no longer need it, and delete the local `.pem`.
6. **Confirm in Billing.** Open **Billing and Cost Management** → **Free tier** and **Bills**. Charges can take up to 24 hours to appear, so **check again tomorrow**.

Keep your VPC, subnets, Security Group, IAM users, and roles — they are free and Lab 4 uses them.

Module 4: Relational Database Service (RDS)

4.1 Why Managed Databases

Amazon RDS runs relational databases for you: provisioning, OS and engine patching, automated backups, point-in-time recovery, failover, and monitoring. Compare that with installing MySQL on an EC2 instance, where every one of those tasks is yours to build and maintain.

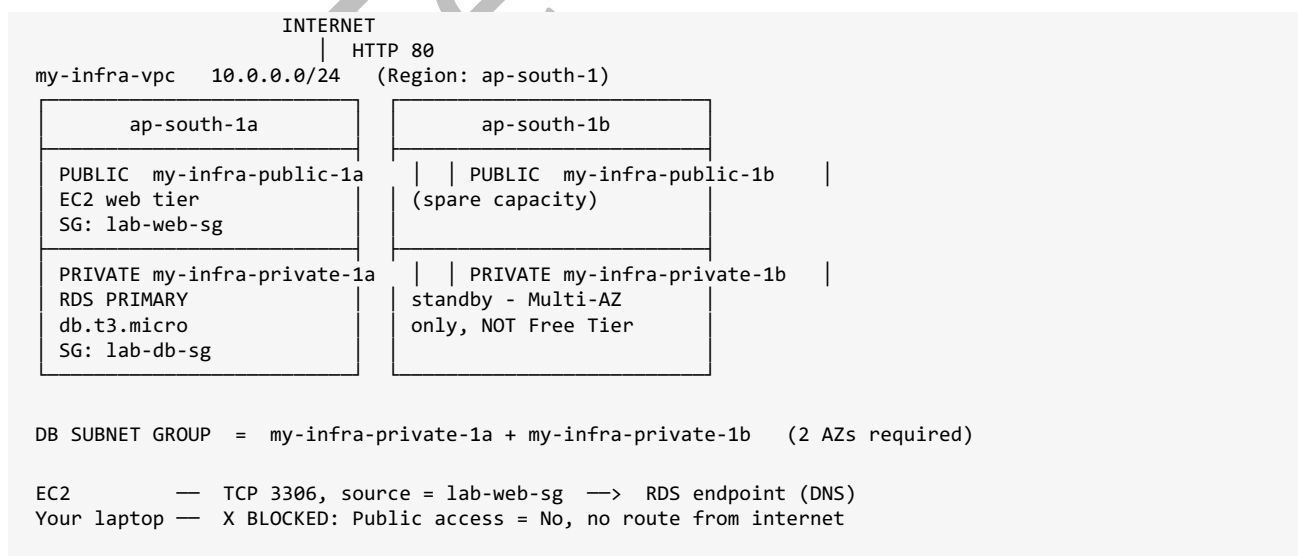
Supported engines: MySQL, PostgreSQL, MariaDB, Oracle, SQL Server, and **Amazon Aurora** (MySQL- and PostgreSQL-compatible).

The trade-off is control: you get no OS-level access and no sudo. You tune the database through **parameter groups** rather than editing config files. For the overwhelming majority of workloads, that trade is worth it.

4.2 RDS Building Blocks

Component	Purpose
DB instance	The compute + memory running the engine, sized by instance class (e.g. db.t3.micro)
DB subnet group	A collection of subnets — in at least 2 AZs — that RDS may place instances in. Always private subnets.
Parameter group	Engine configuration settings (the equivalent of my.cnf)
Option group	Optional engine features (e.g. Oracle/SQL Server extras)
Security Group	Firewall controlling who may reach the DB port
Endpoint	The DNS name your application connects to — always use the endpoint, never an IP
Automated backups	Daily snapshot + transaction logs enabling point-in-time recovery
Manual snapshots	User-initiated backups that survive instance deletion

Diagram 4.2 — Where an RDS database belongs



A DB subnet group is mandatory and needs subnets in two or more AZs — even for a Single-AZ database. This is exactly why Lab 1 built two private subnets.

4.3 High Availability: Multi-AZ vs. Read Replicas

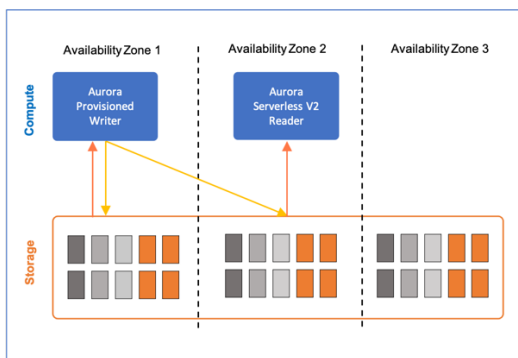
Students routinely confuse these. They solve different problems and can be used together.

	Multi-AZ Deployment	Read Replica
Solves	High availability / disaster recovery	Read scalability
Replication	Synchronous to a standby	Asynchronous
Can you read from it?	No — the standby is idle (Multi-AZ <i>instance</i> deployment)	Yes — that's the point
Failover	Automatic, typically 60–120 seconds	Manual promotion
Location	Different AZ, same Region	Same AZ, cross-AZ, or cross-Region
Free Tier	No — roughly doubles cost	No

Production guidance: Multi-AZ for anything that matters, plus read replicas if reads dominate. **For this lab, choose Single-AZ** to stay free.

4.4 Amazon Aurora: Clusters, Writers & Readers

Amazon Aurora is AWS's cloud-native, MySQL- and PostgreSQL-compatible engine. Its architecture differs fundamentally from standard RDS: **compute and storage are decoupled**.



The regional cluster

An **Aurora regional cluster** is a single-Region database container holding shared storage plus multiple database nodes.

- **Definition:** a logical grouping of compute and storage located in one specific AWS Region.
- **Storage:** a **virtual cluster volume** spanning **three Availability Zones** with **six-way replication** (two copies per AZ). It auto-grows in 10 GiB increments up to 128 TiB.
- **Components:** **one writer instance** and optional **reader instances** (up to 15).
- **Endpoints:** a **cluster (writer) endpoint** and a **reader endpoint** that load-balances across readers.

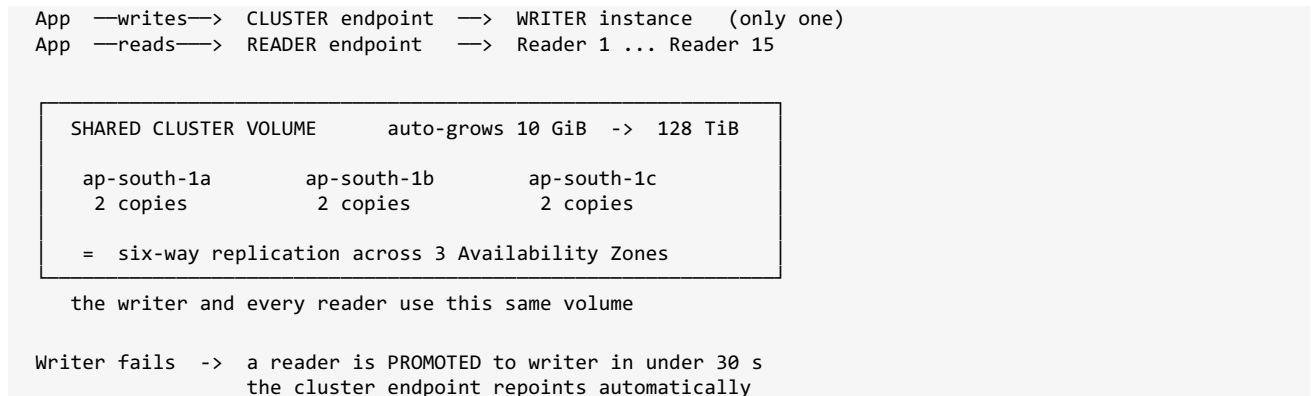
The writer instance

- **Definition:** the primary database compute node running the engine.
- **Capabilities:** handles SQL reads and writes, updates, and schema changes (**DDL**).
- **Failover:** if the writer fails, Aurora **automatically promotes a reader** to become the new writer — typically in **under 30 seconds**, far faster than standard RDS Multi-AZ.
- **Connection:** reached through the **cluster write endpoint**. Point your application at the endpoint, never at an instance address, so failover is transparent.

	Regional Cluster	Writer Instance
What it is	Container of shared storage + nodes	One compute node inside the cluster
How many	One per Region (per cluster)	Exactly one writer at a time

	Regional Cluster	Writer Instance
Handles writes	Via its writer	Yes
Survives node failure	Yes — storage is independent of compute	No — a reader is promoted in its place

Diagram 4.4 — Aurora, with compute and storage pulled apart



Why this design is different from standard RDS: the compute nodes hold no data. All six copies of your data live in the shared cluster volume (**2 copies × 3 AZs = six-way replication**), so losing a writer costs you a promotion, not a restore. It also explains the endpoints — point your writes at the **cluster endpoint** and your reads at the **reader endpoint**, and failover becomes invisible to your application.

Beyond the basics: Aurora Serverless v2 scales capacity automatically in fine-grained increments, and **Aurora Global Database** replicates to secondary Regions with typically sub-second lag.

Hands-On Lab 4: Free Tier Guide

Goal: Launch a Free Tier MySQL database into the private subnets of `my-infra-vpc`.

🎯 Free Tier settings — the ones that matter

Setting	Choose exactly this	Why
Engine	MySQL (or PostgreSQL / MariaDB)	✅ Free Tier eligible. Aurora, Oracle, SQL Server EE are not
Templates	Free tier	This single choice pre-selects most safe options
DB instance class	db.t3.micro or db.t4g.micro	750 DB-instance-hours/month . db.t4g (Graviton) is cheaper beyond Free Tier
Deployment option	Single-AZ DB instance	Multi-AZ roughly doubles cost
Storage type	General Purpose SSD (gp3)	
Allocated storage	20 GB	Free Tier ceiling is 20 GB of DB storage
Storage autoscaling	Disable	Prevents silent growth past 20 GB
Public access	Yes	'No' in production
Performance Insights	Disable (or 7-day free retention)	Longer retention is billable
Backup retention	1 day (or 7)	Free Tier includes backup storage up to your DB size
Number of databases	One	750 hours = exactly one instance for a month

Step-by-step (AWS Management Console)

1. Confirm the Region is **Asia Pacific (Mumbai)** **ap-south-1**.
2. **Create the DB subnet group first.** RDS → Subnet groups → Create DB subnet group.
 - **Name:** lab-db-subnet-group
 - **VPC:** my-infra-vpc
 - **Availability Zones:** ap-south-1a and ap-south-1b
 - **Subnets:** select my-infra-public-1a (10.0.0.128/26) and my-infra-public-1b (10.0.0.192/26) → **Create**
3. **Create a database Security Group.** VPC → Security groups → Create security group.
 - **Name:** lab-db-sg, **VPC:** my-infra-vpc
 - **Inbound rule:** Type **MySQL/Aurora (3306)**, **Source:** Custom → select **lab-web-sg** and create
4. **RDS → Databases → Create database.**
5. **Choose a database creation method: Standard create.**
6. **Engine options: MySQL**, latest available version.
7. **Templates: Free tier** ← *the most important click in this lab.*
8. **Availability and durability: Single-AZ DB instance deployment (1 instance).**
9. **Settings:**
 - **DB instance identifier:** lab-mysql-01
 - **Master username:** admin
 - **Credentials management:** Self managed → don't keep autogenerate and **Give the password**
10. **Instance configuration: Burstable classes (includes t classes) → db.t3.micro (or db.t4g.micro).**
11. **Storage:**
 - **Storage type: General Purpose SSD (gp3)**
 - **Allocated storage:** 20 GiB
 - **Enable storage autoscaling: UNCHECKED** 🚫
12. **Connectivity:**
 - **Compute resource:** Don't connect to an EC2 compute resource
 - **VPC:** my-infra-vpc
 - **DB subnet group:** lab-db-subnet-group
 - **Public access:** No 🚫
 - **VPC security group:** Choose existing → lab-db-sg (remove default)
 - **Availability Zone:** ap-south-1a
13. **Database authentication: Password authentication.**
14. **Monitoring: uncheck Enable Performance Insights** (or select the free 7-day retention).
15. **Additional configuration:**
 - **Initial database name:** labdb (without this, RDS creates the instance but no database)
 - **Backup retention period:** 1 day
 - **Enable encryption:** leave **checked** — free, and it cannot be added later
 - **Enable auto minor version upgrade:** checked
 - **Deletion protection:** leave **UNCHECKED for this lab**, so cleanup is easy. Enable it in production.
16. Review the **Estimated monthly costs** panel — it should state Free Tier coverage — then **Create database.**
17. Wait **5–15 minutes** for **Status: Available.**
18. **Explore what you built** (no client needed): open **lab-mysql-01** and read the **Connectivity & security** tab.
Note the **Endpoint** and **Port**, and that **Publicly accessible = No.**
19. **Optional connection test** (requires the sql client(mysqlworkbench) where you can check the connection is fine or not)

Lab 4 Clean-Up Steps — DO NOT SKIP

RDS is the most expensive mistake a student can leave running. A forgotten `db.t3.micro` past Free Tier is roughly **\$12–15/month**, and larger classes cost far more. Its storage and backups keep billing even while the instance is stopped.

1. **Delete the database.** **RDS → Databases →** select `lab-mysql-01` **→ Actions → Delete.**
 - If deletion is blocked, **Modify** the instance and turn off **Deletion protection** first.
 - **Create final snapshot: UNCHECK** for this lab — snapshot storage is billable beyond the Free Tier allowance.
 - **Retain automated backups: UNCHECK.**
 - Check acknowledgement if any to delete without snapshot
 - Type `delete me` to confirm **→ Delete.** Deletion takes a few minutes.
2. **⚠ Delete it as "Stopping" is not enough.** RDS lets you stop an instance for a maximum of **7 days**, after which AWS restarts it automatically — and **storage and backups bill the entire time.** For cleanup, **delete.**

Appendix A: Free Tier Cost Guardrails

Set these up **today**, before your next lab. They take five minutes and have saved countless students an unpleasant surprise.

1. **Enable billing alerts.** **Billing and Cost Management → Billing preferences →** check **Receive AWS Free Tier alerts** and **Receive billing alerts / PDF invoices**, and add your email.
2. **Create a zero-spend budget.** **Billing → Budgets → Create budget → Use a template → Zero spend budget →** enter your email **→ Create budget.** You will be emailed the moment your bill exceeds \$0.01.
 - Add a second **Monthly cost budget** at a small amount (e.g. **\$5**) as a backstop.
3. **Watch Free Tier usage.** **Billing → Free tier** shows current and forecast usage per service. Anything forecast above 100% needs action now, not next month.
4. **Use Cost Explorer weekly.** **Billing → Cost Explorer**, grouped by **Service**, to see what is actually costing money.
5. **Tag everything** with **Project = dvs-lab** so you can filter costs and find orphans.
6. **Set a personal habit:** finish every lab session by checking **EC2 → Instances**, **EC2 → Volumes**, **EC2 → Elastic IPs**, **RDS → Databases**, **RDS → Snapshots**, and **VPC → NAT Gateways** in `ap-south-1`. Then switch the Region selector to a couple of other Regions and look again — stray resources in a Region you forgot about are a classic trap.

The usual suspects behind surprise bills

Resource	Typical cost	How to avoid
NAT Gateway	~\$32–33/month + \$0.045/GB	Don't create one; use a S3 Gateway Endpoint for labs
Unattached EBS volume	~\$0.10/GB-month	Delete Available volumes after terminating instances
Unassociated Elastic IP	~\$3.60/month each (plus a charge for all public IPv4 since 2024)	Release EIPs you are not using
RDS left running	~\$12–15/month for <code>db.t3.micro</code>	Delete, don't stop
Aurora cluster	\$60+/month	Never create one for practice
Manual snapshots	~\$0.05/GB-month	Delete after the lab
Second concurrent instance	Doubles Free Tier hour burn	Run one at a time

Further Reading

- VPC example — web and database servers: <https://docs.aws.amazon.com/vpc/latest/userguide/vpc-example-web-database-servers.html>
- **Homework** — VPC with private subnets and NAT: <https://docs.aws.amazon.com/vpc/latest/userguide/vpc-example-private-subnets-nat.html>
- AWS Regions and Availability Zones: https://aws.amazon.com/about-aws/global-infrastructure/regions_az/
- Amazon Aurora pricing: <https://aws.amazon.com/rds/aurora/pricing/>
- Microsoft data centre tour (how a data centre physically looks): <https://datacenters.microsoft.com/globe/explore/datacenter>

A final word

"Success is the sum of small efforts, repeated day in and day out." — Robert Collier
"The future depends on what you do today." — Mahatma Gandhi

Build something small every day, break it deliberately, then clean it up. That loop — build, break, fix, clean up — is how cloud engineers are actually made.

Happy building!